Time-evolving SED Gaussian Process — v5 Writeup

Change log and rationale for collaborator handoff

May 2026

Contents

1. What you're getting

Three scripts plus a comparison harness:

- `gp_utils.py` — `KernelConfig` dataclass, kernel/mean factories, point classification.
- `run_gp.py` — CLI to train, predict on `X_fill`, post-process, save `runs/<tag>/`.
- `plot_results.py` — CLI to load a run and produce the diagnostic plots in this document.
- `compare_runs.py` — side-by-side overlay of any two runs.

The "production" run is `runs/matern52_addw_addt_linear_opt_v5/`. Everything below describes that configuration unless noted otherwise.

2. Final v5 configuration on one page

Component	Choice
Wavelength kernel	Matern 5/2, *additive* (short + long)
Time kernel	Matern 5/2, *additive* (short + long)
Mean function	`LinearNDInterpolator` on prior points; `NearestNDInterpolator` outside hull
Per-class jitter	$\sigma_{\rm phot} \geq 0.012$, $\sigma_{\rm spec} \geq 0.005$ (both binding at floor)
Optimization	L-BFGS-B on a 2500-point subsample, then final `gp.compute` on full N=8832
Early-time rise	tanh-blended C^∞ extrapolation for $\log t < -4$
Early-time spec	cumulative-min in wls for $\log t < -4$ ("blue" / hot-BB-like)

Final hyperparameters (after optimization on the subsample; $\ell = \sqrt{\text{metric}}$):

Parameter	Value	Notes
`amp`	0.0135	overall kernel amplitude
`sigma_phot`	0.0120	at floor
`sigma_spec`	0.0050	at floor
`metric_w`	0.0258	short wls; $\ell \approx 0.16$
`metric_w2`	5.90	long wls; $\ell \approx 2.43$
`weight_w_short`	0.003	kernel is effectively color-only
`metric_t`	0.126	short time; $\ell \approx 0.35$
`metric_t2`	7.23	long time; $\ell \approx 2.69$
`weight_t_short`	0.027	kernel is overwhelmingly long-time

Fit quality on the full training set:

Metric	Phot (n=631)	Spec (n=8201)	Total
χ^2/N	0.42	0.86	0.83
Within $\pm\sigma_{\rm eff}$	—	—	95.0%

Total runtime $\approx$ 140 s on this laptop (optimization 69 s + final compute 5 s + prediction 51 s).

3. Final results

3.1 Posterior mean

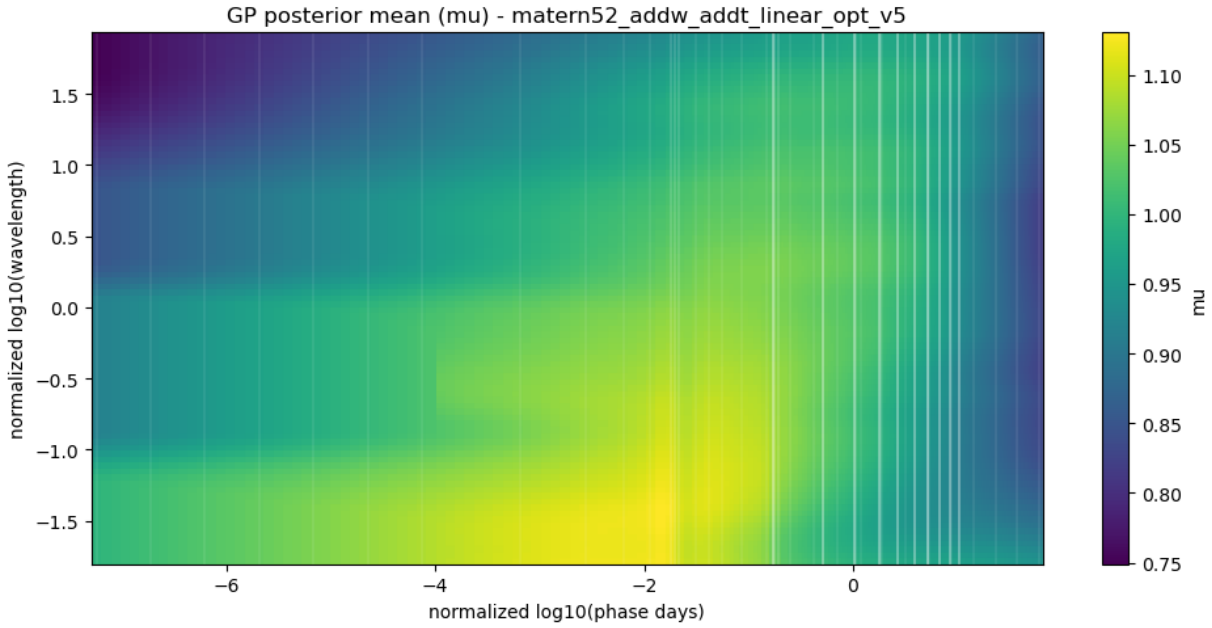

Figure 1: GP posterior mean over the (log-wavelength, log-phase) grid. Vertical white bands mark the unique training phases (densely clustered around peak; sparse and irregular in the tails).

3.2 Posterior std

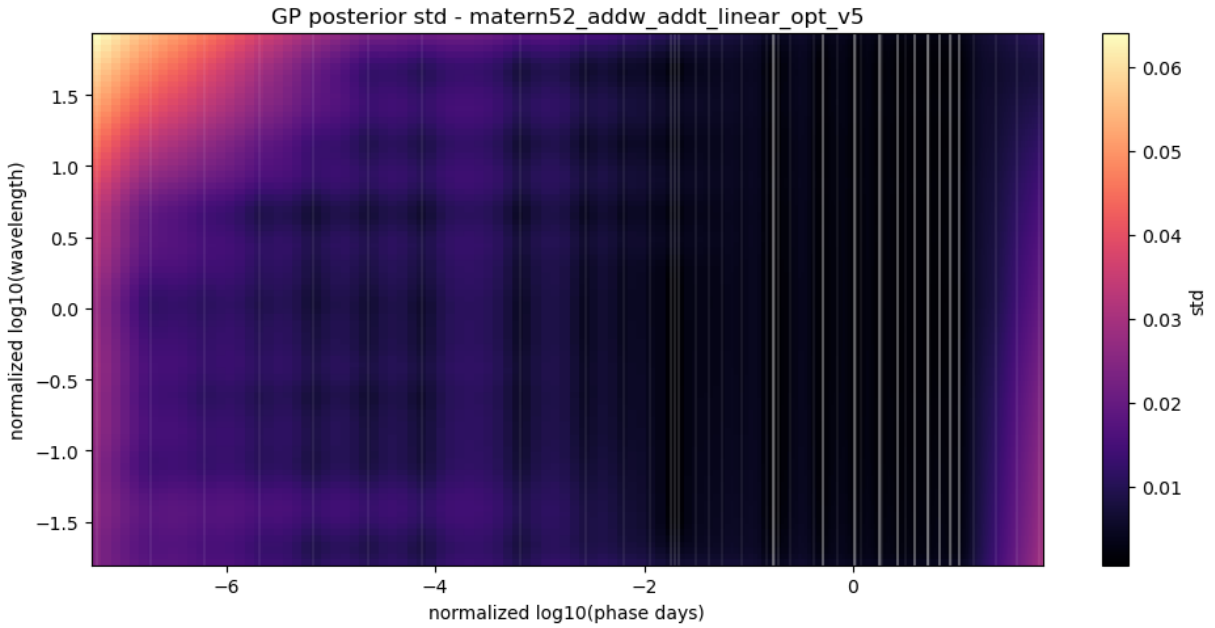

Figure 2: GP posterior std. Small near training (~0.005–0.01), inflates to ~0.04–0.06 outside coverage as expected from a properly conditioned GP.

3.3 Phase profiles at six wavelength slices

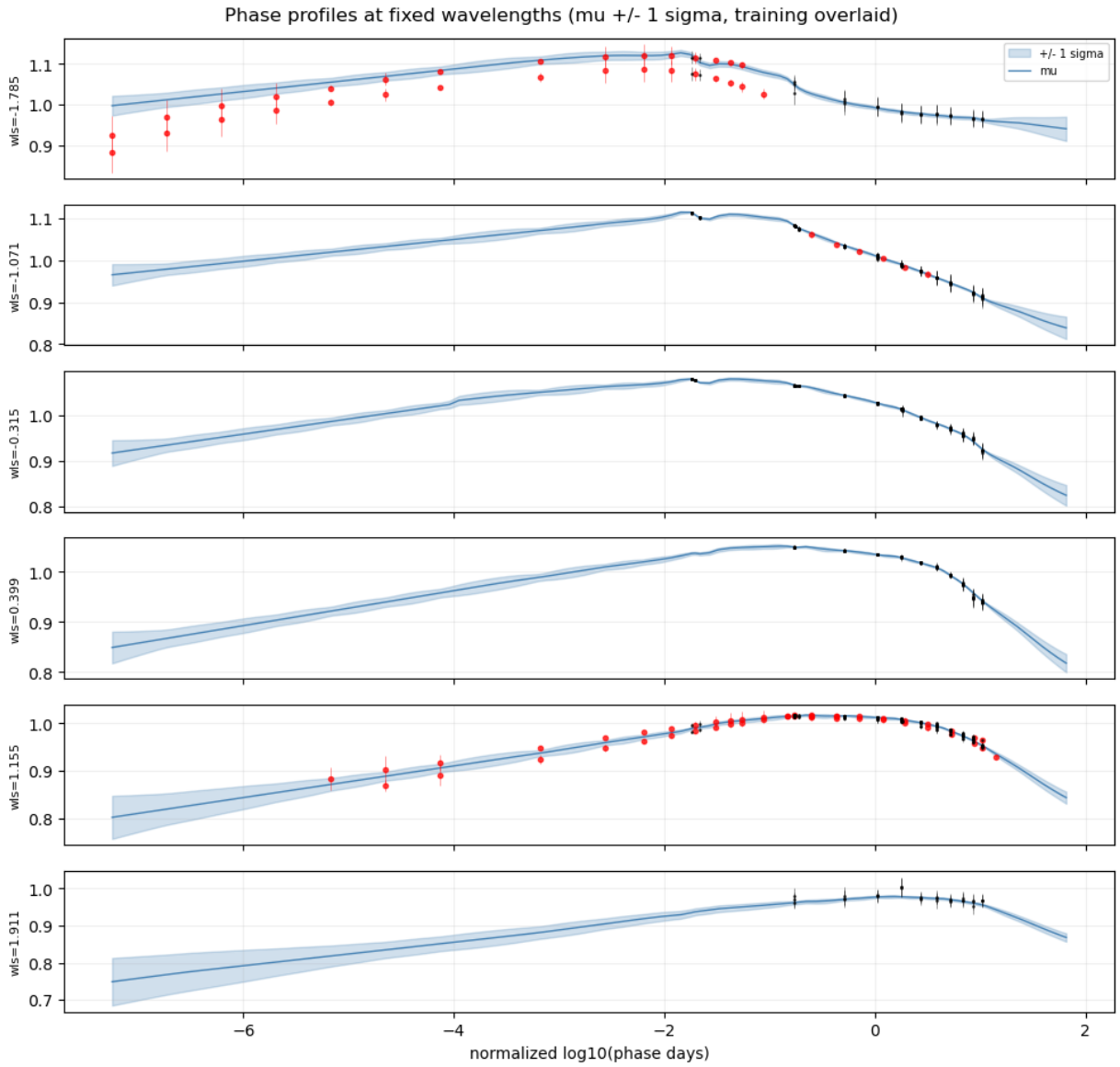

Figure 3: GP mean $\pm 1\sigma$ vs phase, training points overlaid (phot=red, spec=black). Each panel takes a fixed wavelength slice through the heatmap. The $\log t = -4$ region is C^∞-smooth across the constraint join; data clusters at $\log t > -1$ are smoothly traversed without zig-zagging.

3.4 Real-spectrum snapshots

For each requested phase (the script default is $-2, -1, 0, 0.5, 1$ in normalized log-phase), the spectrum panel:

1. Snaps to the *nearest spec training phase* (so we look at an actual measured spectrum).
2. Overlays *every* other spec training phase within ± 0.05 of that chosen phase — there are several "near-simultaneous" spec sets in the bundle that don't overlap in wavelength coverage.
3. Plots the GP $\pm 1\sigma$ at the snapped phase.
4. Adds any phot points within the same window, in red.

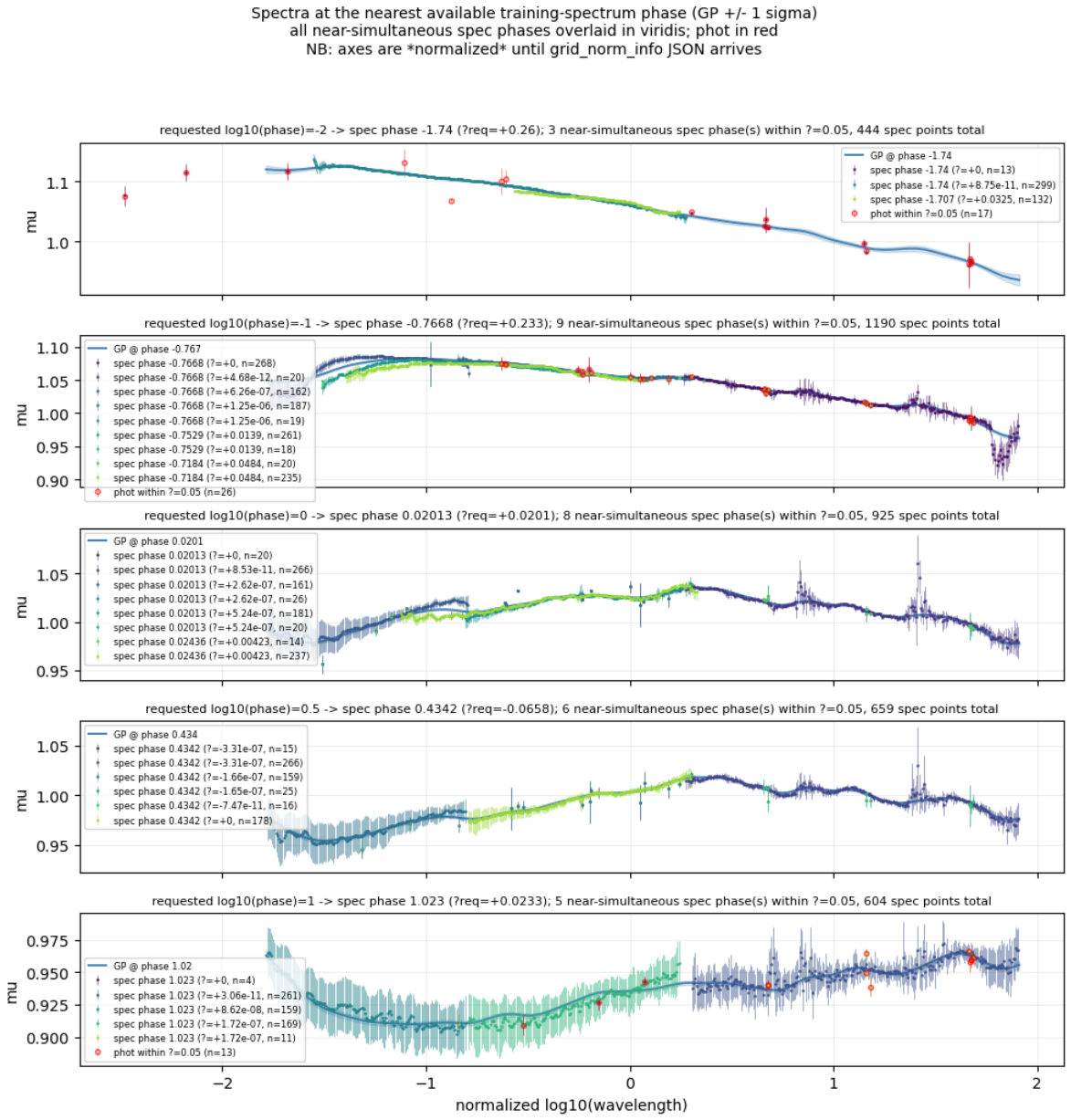

Figure 4: Spectrum panels with all near-simultaneous spec phases overlaid. The legend reports each phase's Δ from the chosen one and the point count. The bottom panel ($\log t = 1$) is well into the late-time regime; the top panel ($\log t = -2$) is close to peak.

3.5 Training residuals

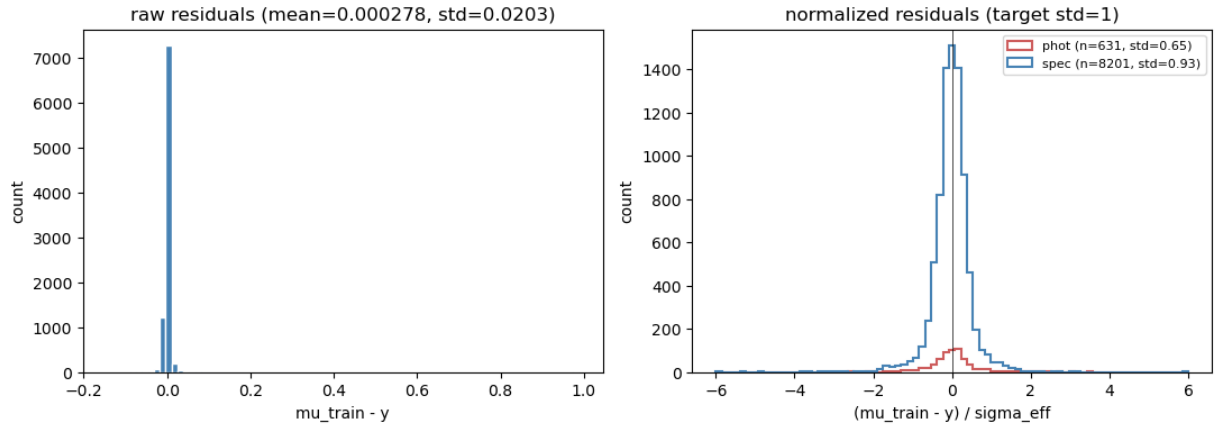

Figure 5: Residuals $(\mu_{\text{train}} - y)/\sigma_{\text{eff}}$, split by class. Phot is moderately under-fit (variance ~0.4), spec is close to unity (~0.9). The spec tail is mildly broader, consistent with the floor.

3.6 Training coverage (for reference)

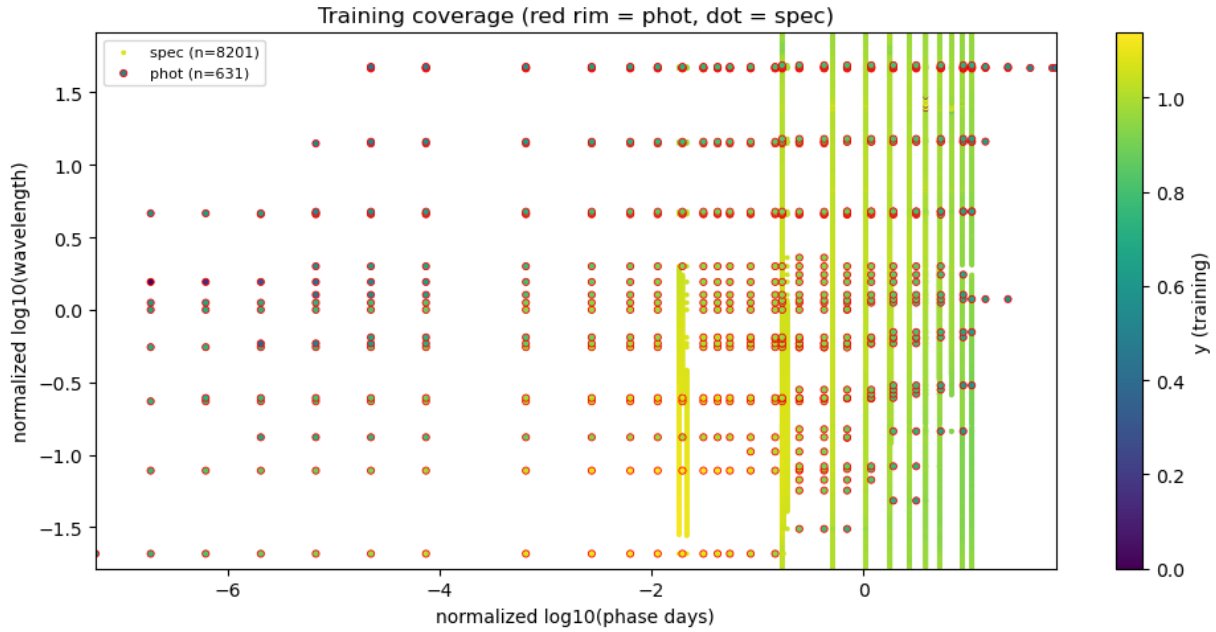

Figure 6: Where the data live in the (log-wls, log-phase) plane, colored by class. Phot is sparse and broadly distributed in time; spec is in dense temporal clusters with rich wavelength coverage.

4. Why the kernel and constraints look the way they do

4.1 Additive kernels on both axes

The dataset has structure on two physically distinct scales in each dimension:

- **Wavelength.** Color varies smoothly across the whole spectrum (long scale, $\ell \sim 1$–2 in normalized log-wls) while spectral features (lines, blue/red components) live at short scales ($\ell \sim 0.05$–0.2). A single-scale Matern can't do both.
- **Time.** Within an observing cluster the SED changes slowly compared to the cluster's internal phase spread (short scale $\ell \sim 0.2$–0.4); the model also has to bridge multi-day gaps between clusters (long scale $\ell \sim 2$–3).

We sum two same-family kernels with different metrics on each axis:

$$k_{\text{axis}}(\Delta x) = w_{\text{short}}\, k(\Delta x;\, \ell_{\text{short}}) + (1 - w_{\text{short}})\, k(\Delta x;\, \ell_{\text{long}}),$$

with george's `metric` $= \ell^2$. The full 2-D kernel is the product `k_wls * k_time`, scaled by `amp`.

After optimization, `weight_w_short = 0.003` and `weight_t_short = 0.027` — the optimizer is almost entirely on the long scales. This is a *consequence* of the per-class jitter floors below: without the floors, the optimizer earns likelihood by zigzagging through near-simultaneous spec scatter on the short scales (see §5).

4.2 Per-class jitter floors

We model the diagonal as

$$\sigma^2_{\text{eff}}(i) = \text{yerr}^2(i) + \begin{cases} \sigma^2_{\text{phot}} & i \in \text{phot} \\ \sigma^2_{\text{spec}} & i \in \text{spec} \end{cases}$$

and let the optimizer fit `sigma_phot` and `sigma_spec`. The crucial detail is that *both* parameters have a lower bound:

- `sigma_phot >= 0.012` — ~1% calibration floor; smaller values let the GP overfit phot scatter.
- `sigma_spec >= 0.005` — the typical near-simultaneous-spec internal disagreement; smaller values let the GP zigzag through every individual spec point.

Both floors are binding in v5. Without them the optimizer drives both to $\sim 10^{-5}$, which is the v3/v4 regression we hunted (Appendix A).

Classification of training points into phot vs spec is heuristic: a unique training phase with ≥ 50 wavelengths is "spec", otherwise "phot". The threshold is `--phot-spec-threshold` (default 50) and the resulting class array is saved with the predictions.

4.3 Linear mean function (with nearest fallback)

A piecewise-constant mean (e.g., `NearestNDInterpolator` alone) creates artificial discontinuities at the prior-point Voronoi-cell boundaries. We instead use `LinearNDInterpolator` over the supplied prior points and fall back to `NearestNDInterpolator` for query points outside the prior convex hull. The interpolator pair is built once per session and cached to `prior_linear_interp.pkl` (~85 MB).

4.4 Optimization on a subsample

Cholesky scales as $\mathcal{O}(N^3)$; on the full N=8832, each `gp.compute` is ~20–30 s and L-BFGS-B does ~150 evaluations. We optimize on a 2500-point stratified random subsample (all 631 phot kept, 1869 spec random), which finishes in ≈ 1 minute. We then run a single final `gp.compute` on the full data with the optimized hyperparameters before predicting.

The Cholesky-only check confirms this is fine: the log-likelihood at the full-data compute (29320) is much higher than the subsample log-likelihood (8571), as expected, and the subsample-optimized hyperparameters generalize cleanly — χ^2/N on the held-out spec is 0.86.

4.5 Early-time monotonic rise (tanh-blend)

For $\log t < -4$ (i.e., the unmeasured pre-explosion / very-early region), we want the model to rise smoothly toward zero rather than oscillate at the GP prior. Per wavelength column:

1. Take the GP value `mu_cutoff` and slope at the first grid phase ≥ -4. The slope is a linear-fit slope over the next 5 grid points (forced $\geq$ `min_slope = 0.005` so the extrapolation is always increasing).
2. Define `mu_extrap(t) = mu_cutoff + slope * (t - t_cutoff)` across the whole phase axis, floored at `floor_fraction = 0.5 * mu_cutoff` to prevent absurdly low values when the slope is steep.
3. Blend with the GP via a tanh weight,

$$w(t) = \tfrac{1}{2}\Big(1 + \tanh\left((t - t_{\rm cutoff})/s\right)\Big), \qquad s = 0.3,$$

so $\mu(t) = (1 - w)\,\mu_{\rm extrap}(t) + w\,\mu_{\rm GP}(t)$.

For $t \ll t_{\rm cutoff} - 2s$ the constraint dominates; for $t \gg t_{\rm cutoff} + 2s$ the GP is intact. Because `mu_extrap` matches the GP value *and* slope at the cutoff and w is C^∞, the join is C^∞ — there is no kink and no curvature jump (this was the v4 regression).

4.6 Early-time blue spectrum (cumulative-min in wavelength)

For phases below the same cutoff, we additionally enforce that the spectrum is non-increasing in wavelength (i.e., flux rises toward shorter wls, hot-blackbody-like early emission). We sort each phase column by wavelength and apply `np.minimum.accumulate` from short to long. Note this does *not* enforce a strict blackbody — only monotonicity in wavelength.

5. Diagnosing and fixing the v3 / v4 discontinuities

There were two regressions in v4 that we fixed in v5; the diagnoses motivated the choices in §4.

Cluster wiggles for $\log t > -1$. With `sigma_spec` essentially zero, the optimizer gains likelihood by setting a *short* time-scale (low `metric_t`, high `weight_t_short`) and threading the GP through per-cluster spec-point scatter. The fix is to floor `sigma_spec` at 0.005, which removes that incentive: in v5 the optimizer responds by collapsing `weight_t_short` from 0.110 (v4) to 0.027.

Curvature jump at $\log t = -4$. The v4 monotone-early enforcement was C^1 (matched value and slope but not curvature). The linear extrap has zero second derivative; the GP has a finite one. The eye reads the curvature jump as a kink. The fix is the tanh-blend in §4.5, which is C^∞.

The side-by-side comparison makes both fixes visible:

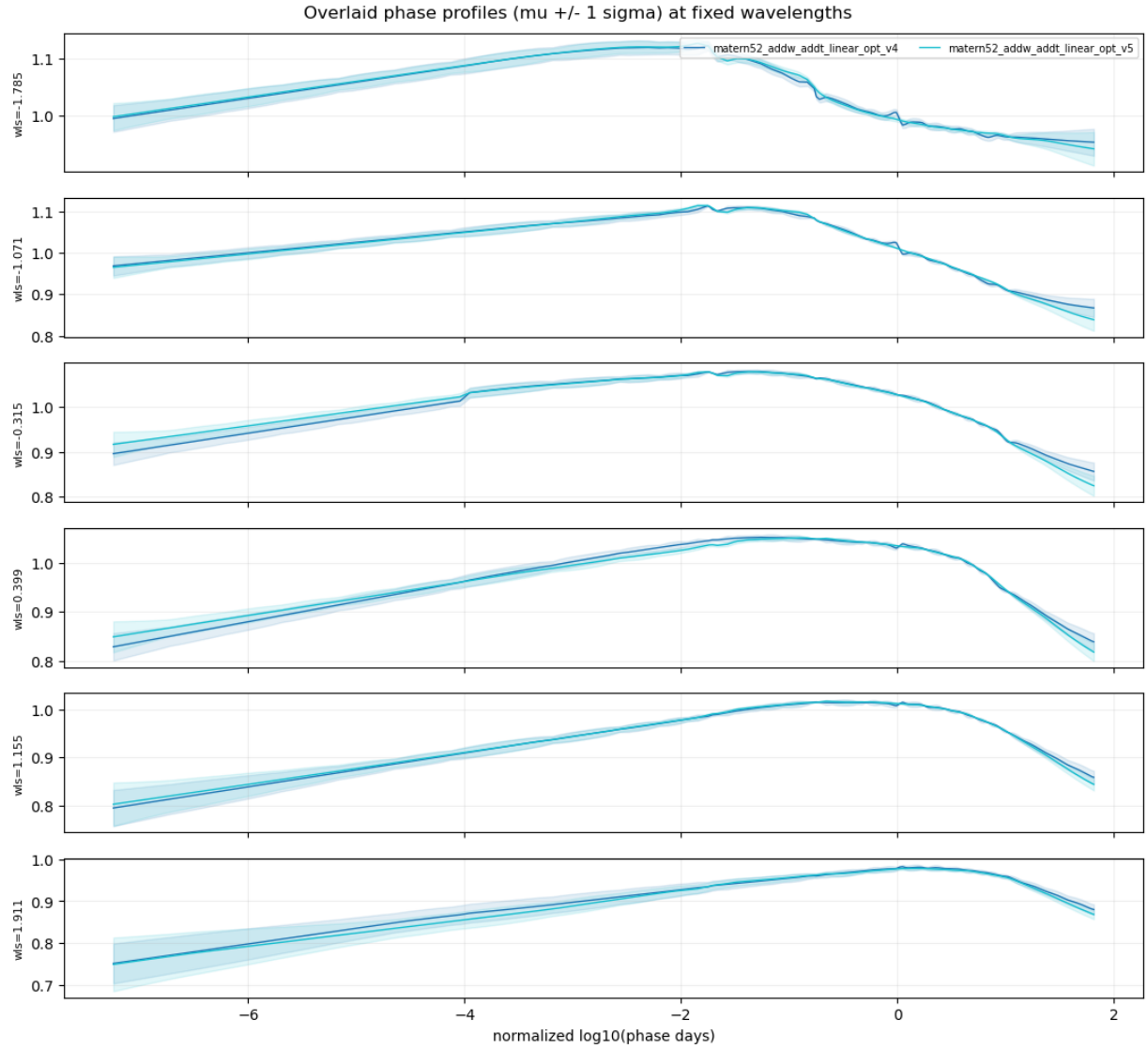

Figure 7: Phase profiles: v4 (dark blue) vs v5 (cyan). The cluster-zigzag fix is most obvious in the top three panels around $\log t \in [-1, 0]$. The curvature-jump fix is most obvious in panels 3, 4, 5, 6 at $\log t \approx -4$, where v4 has a clear kink that v5 doesn't.

6. Open issues and future work

- **Denormalization of plot axes.** `plot_results.py` plots in the *normalized* coordinate frame because `gp_minimal_bundle_meta.json` (with `grid_norm_info`: `x_means`, `x_stds`, `y_mean`, `y_std`) was not in the bundle. The plotting code has identity-fallback functions and a `GRID_NORM_INFO` placeholder; once the meta JSON arrives, swap it in and axes will render in physical units. See `plot_results.py:GRID_NORM_INFO` and the `denormalize_*` helpers.
- **`predict_train` cost.** We currently call `gp.predict` on the training X to compute residuals (~12 s per run). For larger datasets, replace with the algebraic identity $y - \mu_{\text{train}} = \Sigma\,(K + \Sigma)^{-1}\,y$ which the existing Cholesky already provides at zero extra cost.
- **Spec phase de-duplication.** The bundle contains many spec phases at $\Delta < 10^{-7}$ (essentially identical). The `_make_spectrum_figure` overlays them all — fine for diagnostics, but a real "select 5 representative spectra" routine would dedupe by phase + instrument before plotting.
- **Per-band photometric jitter.** A more principled phot model would estimate `sigma_phot` per filter (or per filter-system) rather than globally. Easy to add: extend `KernelConfig` with a per-band parameter and pass a `band_id` array to `compute_diagonal`.
- **Anisotropic short-time scale.** The optimizer settled on `weight_t_short = 0.027` meaning the short scale is essentially unused. If a future user wants real intra-cluster smoothing, raise the lower bound on `weight_t_short` (or set `--no-additive-time` for a single-scale time kernel).
- **Hyperparameter posteriors.** L-BFGS-B gives a point estimate. If you care about hyperparameter uncertainty (e.g., for downstream propagation), wrap `_make_neg_ll` in `emcee` or `dynesty` — the existing code is set up for it (the parameter vector and bounds are already exposed via `KernelConfig.free_param_names`).

Appendix A: iteration log

Tag	Key change	Result
`matern32_nearest_baseline_jitter`	Original collaborator setup: matern32 + nearest mean + tiny global jitter	discontinuities everywhere; baseline
`matern52_linear_opt`	matern52 + linear mean + per-class jitter + L-BFGS-B	smoother, but underfits very early time
`matern52_addw_linear_opt`	adds additive wls (color + features)	small improvement on spectra
`matern52_addw_addt_linear_opt` (v1)	adds additive time	best fit; user approved as "money plot"
`..._v2`	+ cumulative-max early-time monotonic rise	cumulative-max produced flat plateaus; regression
`..._v3`	+ linear-interp early-time + $\sigma_{\text{phot}} \geq 0.012$	introduced cluster-wiggle regression at $\log t > -1$
`..._v4`	+ C^1 slope-matched extrapolation + cumulative-min blue early	cluster wiggles persisted *and* introduced curvature kink at $\log t = -4$
`..._v5`	+ $\sigma_{\text{spec}} \geq 0.005$ + tanh-blend (C^∞)	both regressions resolved; **production**

The two key lessons from the iteration:

1. **Symmetry of jitter floors matters.** Putting a floor on `sigma_phot` alone (v3) doesn't help; the optimizer simply moves the wiggle from phot to spec. Both classes need a floor, and the spec floor is the load-bearing one.
2. **Visual smoothness needs C^∞, not just C^1.** A C^1 join with a curvature jump reads as a kink to a human eye, even if the function is technically smooth. A simple tanh weight kills this for free.

Appendix B: file inventory

```
gp.info                              collaborator's notes on the dataset
gp_minimal_bundle.npz                data: X, y, yerr, X_fill, prior, kernel params
plot.py                              collaborator's original plot script (kept for reference)
prior_linear_interp.pkl              cached LinearND/NearestND interpolators (~85 MB)

gp_utils.py                          KernelConfig, kernel/mean factories, classify_points
run_gp.py                            CLI: train + predict + post-process; writes runs/<tag>/
plot_results.py                      CLI: load runs/<tag>/, write figs/
compare_runs.py                      CLI: side-by-side compare any two runs
README.md                            README + integration guide
WRITEUP.pdf                          this document

runs/<tag>/predictions.npz           X_fill, mu, std, mu_raw, point_class_train,
```

	sigma_eff_train, mu_train
runs/<tag>/config.json	every flag, hyperparameter, and metric for the run
runs/<tag>/figs/	all diagnostic plots